

4月29日日报

本日学习内容

- 1. 折叠cell的剩余部分, 替换UITableView的titleForHead, 替换为button, 来控制cell的展开状态
- 2. 学习了通知传值的简单实现
- 3. 学习了单例传值的简单实现

题目1: [138. 随机链表的复制](#)

138. 随机链表的复制

已解答

中等 相关标签 相关企业 提示 Aa

给你一个长度为 `n` 的链表，每个节点包含一个额外增加的随机指针 `random`，该指针可以指向链表中的任何节点或空节点。

构造这个链表的 **深拷贝**。深拷贝应该正好由 `n` 个 **全新** 节点组成，其中每个新节点的值都设为其对应的原节点的值。新节点的 `next` 指针和 `random` 指针也都应指向复制链表中的新节点，并使原链表和复制链表中的这些指针能够表示相同的链表状态。复制链表中的指针都不应指向原链表中的节点。

例如，如果原链表中有 `X` 和 `Y` 两个节点，其中 `X.random -> Y`。那么在复制链表中对应的两个节点 `x` 和 `y`，同样有 `x.random -> y`。

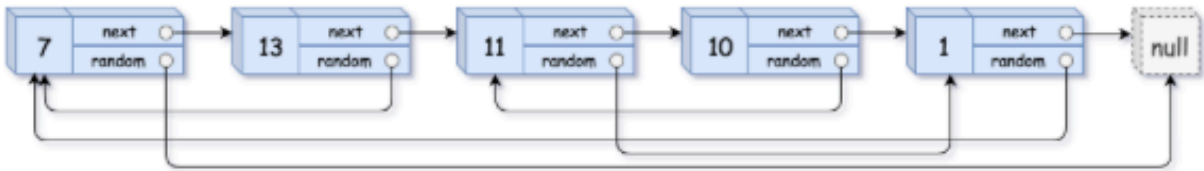
返回复制链表的头节点。

用一个由 `n` 个节点组成的链表来表示输入/输出中的链表。每个节点用一个 `[val, random_index]` 表示：

- `val`：一个表示 `Node.val` 的整数。
- `random_index`：随机指针指向的节点索引（范围从 `0` 到 `n-1`）；如果不指向任何节点，则为 `null`。

你的代码只接受原链表的头节点 `head` 作为传入参数。

示例 1：



输入: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
输出: [[7,null],[13,0],[11,4],[10,2],[1,0]]

```
1 #define ll long long
2 class Solution {
3 public:
```

```
4     Node* copyRandomList(Node* head) {
5         if (head == nullptr) {
6             return nullptr;
7         }
8         unordered_map<Node*, Node*> mp;
9         Node* cur = head;
10        while (cur != nullptr) {
11            mp[cur] = new Node(cur->val);
12            cur = cur->next;
13        }
14        cur = head;
15        while (cur != nullptr) {
16            mp[cur]->next = mp[cur->next];
17            mp[cur]->random = mp[cur->random];
18            cur = cur->next;
19        }
20        return mp[head];
21    }
22 };
```

本日遇到的问题

1. 对于通知传值通知监听的注册写法理解不清
2. 移除观察者和发送通知等方法理解不清

明日学习计划

1. 对学习的UI的知识点梳理, 并对掌握不清的内容多进行练习